goxpyriment — Timing Tests

christophe@pallier.org

2026-07-28

Contents

Timing-Tests: A Guide for Researchers

This document explains how to use the `Timing-Tests` program to characterise the timing behaviour of your computer before running a psychophysical experiment.

> **Quick reference** — flags, equipment, and one-line test descriptions are in `tests/Timing-Tests/README.md`. This document explains the *why*.

> For the EEG/MEG question specifically — how consistent the delay is between a TTL trigger and the physical stimulus — see Minimising trigger-to-stimulus jitter, which gives the measured decomposition and the two things that matter most.

Why timing matters

In a psychology experiment every stimulus has an intended onset and offset time. Whether you are presenting a word for exactly 100 ms, playing a tone that should coincide with a visual flash, or measuring a reaction time to the nearest millisecond, you are trusting the computer to do two things correctly:

1. **Present stimuli when you ask it to.** If you ask for a 100 ms word, does the word actually appear on screen for 100 ms?
2. **Record timestamps accurately.** If you record the time of a key press, is that the time the key was physically depressed, or is it delayed by polling?

Neither is guaranteed. Both depend on your operating system, graphics driver, audio driver, and hardware. This suite lets you measure them on *your specific machine*, so you can report them in your methods section and fix problems before data collection begins.

Read this before you trust any number

The statistics these tests print come from software timestamps. They record when goxpyriment *believes* a flip happened — not what reached the panel.

This is not a theoretical caveat. In July 2026 a presentation bug on a GNOME/Wayland machine left the display showing stale frames for *seconds at a time*, while the program's own numbers stayed textbook-perfect throughout: bright-phase duration 199.915 ± 0.16 ms against a 200 ms target, cycle period 500.05 ± 2.45 ms. Nothing in the console output hinted at a problem. It was visible only with a photodiode. (Cause and fix are documented in `apparatus/CLAUDE.md`; the regression test is `tests/test_clear_only_frames`.)

So:

- **Software statistics tell you about the software.** They are genuinely useful for detecting dropped frames, scheduler jitter, and audio-buffer effects.
- **A photodiode and a trigger box tell you about the experiment.** Only they can confirm that light actually changed when you said it would.

If you are validating a rig for publication, measure it with hardware. The `av` test is built for exactly that.

The six tests

Two groups: what runs on any machine, and what needs equipment.

```
No hardware needed
  check    Go/no-go: can this machine display and make noise at all?
  display  What is my true refresh rate, and how stable are frame intervals?
  latency  How long does SDL hold audio before it sounds?
```

```
Photodiode and/or trigger box
  av       THE stimulus timing test — visual + audio + TTL, every cycle
  vrr      Can I present arbitrary durations, not just multiples of a frame?
  rt       How precise is my reaction-time measurement?
```

Two related tests live in their own directories: `tests/test_gv_sync` (`.gv` playback synchronisation) and `tests/test_dlpio8` (DLP-IO8-G square-wave characterisation).

This document assumes you have built the program:

```
go build -o Timing-Tests ./tests/Timing-Tests
```

Recording system information (`-sysinfo`)

Before running anything, capture a snapshot of the machine. `-sysinfo` prints it and exits — it opens no window, though it does initialise SDL briefly to enumerate displays and audio devices:

```
Timing-Tests -sysinfo
```

```
Machine:    product: Precision 5490  System: Dell Inc.  Type: laptop
System:     Host: mylab-pc  Kernel: 7.0.0-28-generic x86_64  Uptime: 3h 12m
            OS: Ubuntu 26.04  Shell: bash 5.2.37  Desktop: ubuntu:GNOME
CPU:       Model: Intel(R) Core(TM) Ultra 7 165H  Info: 16 cores / 22 threads
Memory:     RAM: total: 30.04 GiB  used: 6.21 GiB (20.7%)
Graphics:  Card: Intel Corporation Meteor Lake-P [Intel Arc Graphics]  Driver: i915
Audio:      Server: PipeWire  v: 1.4.7
Displays:  [0] Built-in display  1920x1200  60.040 Hz  bounds 0,0 1920x1200  [primary]
            video driver: wayland   (the [N] above is the -d N value)
Audio out:  [0] Speaker
            driver: pipewire
Vblank:    default: onsets are timestamped the moment SDL_RenderPresent() returns.
          Where the driver blocks on the retrace, that moment IS the retrace,
          so the timestamp is a hardware instant. On a frame where the present
         returned early instead, the timestamp comes from the pacing schedule.
        available if asked for with GOXPY_VBLANK=on: Linux DRM vblank (card /dev/dri/card1
1 1920x1200@60.0386 Hz, DRM_IOCTL_WAIT_VBLANK, hardware-verified)
```

The `Vblank` line

It reports two independent things: where this run's onset timestamps will come from, and whether the machine has a kernel vblank clock at all.

By default `FlipTS` returns whichever anchor the driver made available that frame: the present's own return where `SDL_RenderPresent` blocks on the retrace, and a timestamp derived from the pacing schedule where it does not (the run's `Frame pacing` block says which — see The `Frame pacing` block). `GOXPY_VBLANK=on` instead anchors every frame on the kernel's vblank stamps — **off by default, and there is currently no reason to turn it on.**

Measured with a photodiode against a TTL, 1010 cycles per run, on a Raspberry Pi 4 (V3D/kmsdrm) and a Radeon Pro W5700 (radeonsi, X11 exclusive fullscreen):

arm	flip → photon slope	one-frame errors
pacing schedule (5 runs)	−1.62 … +0.12 ppm	none in any run
kernel vblank (1 run)	+0.48 ppm	none

The two are indistinguishable and both are flat to well under a ppm, so the default is the one with five runs behind it on two machines rather than one.

The case the vblank anchor exists for is a host whose nominal refresh rate is badly wrong, where a schedule advancing on it walks away from the panel. Neither machine above is that host — the W5700's nominal is 5.9 ppm from true, or 0.2 ms over an eight-minute block. If you have a rig where the two disagree by much more, this is the switch to try, and `sys vblank_resolution:` in the run's `-info.txt` is how to check it behaved.

If you enable it, read that line. It reports how each frame's vblank was resolved, and it is not decoration: on the W5700 the flip query beat the vblank interrupt on **31.3 % of frames**, and identifying which vblank a frame belongs to is the whole job. A wrong answer there is exactly one frame out, which on a frame grid still looks like a perfectly regular train — so nothing in the timestamps themselves would tell you.

The same line ends with the check that the vblanks came from the **right display**:

```
sys vblank_resolution: frames=30000 … measured=59.9514 Hz nominal=59.9506 Hz (frame period
13 ppm, matches the display)
```

`measured` is the cadence of the vblanks the run actually read, taken from the kernel's own stamps and divided by the vblank *count* so a dropped frame cannot stretch it. `nominal` is the display the experiment drew on. On a single-head machine the two agree by construction and there is nothing to think about. On a laptop with an external monitor they are the sentence worth reading, because the two heads run different clocks and the vblank ioctl names a pipe by index, not by monitor.

That mistake was made and measured. On a Precision 5490 driving a U2720Q on DP-1, with the internal panel also lit, the backend read the internal panel's CRTC — 60.0386 Hz against the external monitor's 59.9514, **1449 ppm apart**. The presents were flawless (the photodiode saw an unbroken 30-frames-per-cycle lock with 3 µs stability for 8.4 minutes) but the recorded onsets walked 24.2 µs per frame until they were a whole frame out and jumped back — 44 times in the run, a flip→photon lag sawtoothing across a full frame, and `onset_source` reporting `hardware-verified` in all 1010 rows. It was the right kind of hardware and the wrong piece of it.

The backend now reads the card's mode resources and picks the CRTC whose programmed mode is the display you are presenting to, names that head in the `Vblank` line (`crtc 1 driving DP-1 2560x1440@59.9514 Hz`), refuses to start if no lit CRTC matches — falling back to the default present-return anchoring, which cannot pick the wrong display — and keeps checking the live cadence for the whole run. If it ever

says `WRONG DISPLAY`, the onsets in that file are on another monitor's grid and should not be used.

Check this line **before** committing to a photodiode capture rather than afterwards in the run's `-info.txt`: a machine with no backend and a run that never opted in produce identical data, and an A/B whose two arms ran the same way looks like a null result rather than a switch that never took.

Keep it alongside your results:

```
Timing-Tests -sysinfo > sysinfo-$(hostname)-$(date +%Y%m%d).txt
```

Reviewers routinely ask for CPU model, OS version, graphics driver, and audio server. Every data file also carries this in its `-info.txt` header, including the **display the window actually opened on** — read it from there rather than assuming, especially on a multi-monitor machine.

Understanding the display refresh cycle

Most monitors refresh at a fixed rate — typically 60 Hz (a new frame every 16.67 ms), 120 Hz, or 144 Hz. Your graphics driver synchronises presentation to this cycle (VSYNC). Two consequences:

- **Durations are quantised to multiples of the frame period.** At 60 Hz you can show a stimulus for 16.67, 33.33, or 50 ms — but not 25 ms. Plan accordingly, or see the `vrr` test.
- **Onset is not when you call the flip; it is the next VSYNC**, anywhere from 0 to 16.67 ms later. goxpyriment keeps that offset constant once the pipeline is warm, but the first frames should be excluded — `-warmup` does this.

And the screen does not change all at once

An LCD paints top to bottom. On the rig this suite was developed against, a square at the bottom of the screen lights **13.15 ms** after one at the top — close to a full frame period. Measured on a 1280×1024 @ 60.02 Hz panel, that is **15.96 µs per pixel row**.

This matters more than it sounds. A stimulus at screen centre appears ~6.6 ms after one at the top. If your photodiode sits in a corner but your stimulus is central, your measured onset is biased by that amount — a systematic error larger than most of the jitter people worry about.

The `av` test draws five squares (four corners plus centre) precisely so you can measure this on your own display. Put a photodiode on a top square and another on a bottom one; the difference is your panel's scan-out gradient. Two squares on the same row are only microseconds apart and serve as a sanity check. Each display needs its own figure.

No hardware needed

`check` — does anything work at all

```
Timing-Tests -test check
```

Flashes a white screen for a second, then plays two sounds. If you do not see the flash or hear both sounds, stop and fix that first. Common causes: the window opened on the wrong monitor (`-d`), the audio device is muted, or the default output is not your speakers.

Measures nothing. It exists so you do not waste a session discovering the hardware was not plugged in.

`display` — true refresh rate and frame stability

```
Timing-Tests -test display -duration-s 30
```

Flips a uniform screen for `-duration-s` and reports the distribution of frame-to-frame intervals, plus the refresh rate implied by their mean. Use that measured rate for `-hz` in later tests rather than assuming 60.

What to look for: a tight distribution centred on your nominal frame period. A long right tail means dropped frames — a compositor, a background process, or power management. A *bimodal* distribution usually means the display is not running at the rate it reports.

The `Frame pacing` block Below the interval statistics the test prints a second block. **It does not report dropped or missed frames** — every present is counted in it, and a frame that never reached the panel would show up in the intervals above, not here.

It answers a different question: *what were this run's flip timestamps anchored to?* `SDL_RenderPresent` is supposed to block until the retrace, but under triple or mailbox buffering it queues the frame and returns immediately. When it does, `Screen.Update` holds the frame to the boundary itself — and the onset it reports is then a *scheduled* time rather than a hardware one. The counts say which happened:

line	meaning	the onset `FlipTS` reports
`blocked`	present covered the frame and returned at, or just inside, the boundary	the present's own return — a hardware instant
`early`	the part of `blocked` that returned *just* inside the nominal boundary	unchanged: still the present's return
`paced`	present came back with most of the frame left, so `Update` held it	the scheduled boundary — synthesised

line	meaning	the onset `FlipTS` reports
`vblank`	a kernel vblank timestamp was available (`GOXPY_VBLANK=on`)	the vblank — measured

A typical good result, on an Intel/Mesa laptop under Wayland:

```
── Frame pacing ──────────────────────────────────
  presents: 1798   (frame = 16.661 ms)
 blocked : 1798 (100.0 %)  — present carried the frame; its return is the onset
           of which 1776 came back inside the nominal boundary by mean
           0.676 ms, max 1.140 ms — the phase offset between the nominal
           frame grid and the panel's, not a wait.
  paced   : 0 (0.0 %)  — returned early; Update held to the schedule
  verdict : the driver blocks. Flip timestamps carry the display's own
           instant, and cannot drift against the panel.
```

The `early` count is not a fault, and a large one is not a warning. A blocking present returns one *panel* period after the last one, while the boundary it is compared against is one *nominal* period after — and the two grids are never in exact phase. On the machine above the gap was 0.676 ms of a 16.661 ms frame, meaning present had blocked for 15.99 ms of every frame. It is a constant offset, re-established by the hardware on every frame, so it cannot accumulate and it does not enter any duration or reaction time you compute (both are *differences* between timestamps, and the offset cancels).

What each verdict means for your experiment:

- **`the driver blocks`** — nothing to do. Onsets are hardware-anchored.
- **`onsets come from the kernel's vblank timestamp`** — nothing to do; this is the most accurate configuration available.
- **`the driver does NOT block`** — your frames are still correctly paced and your *durations* are fine, but the onsets are stamped with a schedule running at the nominal refresh rate. If that rate is wrong, absolute onsets slide against the panel over long blocks. Compare the estimated refresh rate printed above against the nominal one, run `timing-drift` on a photodiode capture before quoting absolute onsets, and consider fullscreen or a session without a compositor (see Improving timing on your system).
- **`MIXED`** — some frames took each path. Worth a photodiode check before quoting absolute onsets.

For a rig whose durations and reaction times matter to a few milliseconds, only the third verdict asks anything of you, and even then it bounds a *slow slide* over minutes, not per-trial error.

`latency` — audio pipeline delay

```
Timing-Tests -test latency
Timing-Tests -test latency -audio-frames 256 -drain-reps 20
```

Plays tones of known duration and spin-polls until the audio stream has drained, measuring how long SDL holds PCM beyond the nominal duration. That difference is your audio output latency.

No hardware needed — but note it measures the *software* pipeline, not speaker-to-microphone delay. For true acoustic onset, use `av` with a microphone.

Photodiode and/or trigger box

`av` — the stimulus timing test

This is the one that matters. Every cycle presents all three modalities at once:

- **Visual** — five squares (four corners + centre) bright for `-frames-on` frames, then dark for `-frames-off` frames
- **Audio** — a tone lasting `frames-on × frame-period`, synchronised to the visual onset or offset from it by `-soa-ms`
- **TTL** — a `-trigger-ms` pulse at the visual onset, on the device chosen by `-trigger-device` (see below)

```
# 200 ms on, 300 ms off at 60 Hz, 100 cycles — the defaults
Timing-Tests -test av -cycles 100

# Visual only, no hardware at all
Timing-Tests -test av -no-sound -no-ttl

# Audio leading the flash by 50 ms
Timing-Tests -test av -soa-ms -50
```

`-no-sound` and `-no-ttl` drop a modality. This is how one test covers what used to be three: `-no-sound` is a pure visual-onset test, and the audio channel of a recording gives tone-onset jitter over a long session.

What to record. A photodiode on one square, the TTL line on a second channel, a microphone if you are checking audio. The quantity of interest is the *offset* between the TTL edge and the luminance step, and how stable it is across cycles. A constant offset is a calibration you subtract; a varying one is jitter you cannot.

What the console tells you. Bright-phase duration, cycle period, and (with sound) audio-queued minus visual onset. All software-side — cross-checks, not ground truth. See the warning at the top of this document.

Reference numbers. Dell Precision 5490, external 1280×1024 @ 60.02 Hz display under Wayland: 100/100 cycles clean, TTL→photodiode 29.878 ± 0.628 ms (from cycle 10 on), top-to-bottom gradient 13.150 ± 0.221 ms, ~5 dropped frames per 100 cycles. Same machine on a bare console with `SDL_VIDEODRIVER=kmsdrm`: 58/58 cycles,

TTL→photodiode 21.51 ± 1.19 ms, **zero** dropped frames. Compositing costs roughly one extra frame of latency and a few percent of dropped frames.

Expect a warm-up transient. The first six cycles on that rig ran 36–39 ms before settling to ~27 ms. `-warmup 10` excludes them from the test's own statistics, but offline analysis tools do not know about it — quote steady-state numbers, and say which cycles you used.

Audio onsets are quantised by the buffer. With `-audio-frames 256` at 44100 Hz, tone onsets land on 5.8 ms steps. This appears as a lattice in the inter-onset intervals and is not jitter — it is the buffer. Halving the buffer halves the step, at the cost of underrun risk.

Choosing a TTL output device

`-trigger-device` selects among five back-ends. They are **not** interchangeable. The trigger is fired immediately after the flip returns, so whatever the write costs falls between the flip and the TTL edge — and it shows up in the trial-to-trial *spread* of that interval, not as a constant offset you could subtract afterwards.

`-trigger-device`	Path to the pin	Logic	Extra flags
`dlpio8` (default)	USB serial (FTDI)	5 V	`-port`
`parallel`	`ioctl` on ppdev	5 V	`-parallel-port`
`gpio`	`ioctl` on a GPIO chip	**3.3 V**	`-gpio-chip`, `-gpio-pins`
`ft232h`	USB bulk transfer (MPSSE via usbfs)	**3.3 V**	— (first device found)
`labjackt4`	Modbus TCP over the network	**3.3 V**	`-labjack-host` (required)

`parallel` and `gpio` are both a local `ioctl` and carry none of the USB frame scheduling that governs the DLP-IO8's output latency (see the DLP-IO8 section below — the FTDI latency timer does *not* fix that). Prefer `parallel` on a desktop that still has an LPT port, and `gpio` on a single-board computer. `ft232h` and `labjackt4` are for a rig that has neither: both put a link back between the flip and the edge — USB for the FT232H, an Ethernet round trip for the T4 — so they are a fallback, not an improvement on the two `ioctl` paths.

How much each costs here has **not been measured**; the ordering above is what the transport implies, not a result. The TTL channel of the recording is the only thing that settles it for a given machine.

```
Timing-Tests -test av -trigger-device parallel                    # LPT, auto-detect
Timing-Tests -test av -trigger-device parallel -parallel-port /dev/parport1
Timing-Tests -test av -trigger-device gpio                        # /dev/gpiochip0, de
Timing-Tests -test av -trigger-device gpio -gpio-chip /dev/gpiochip4 -trigger-pin 2
Timing-Tests -test av -trigger-device ft232h                      # first FT232H on th
Timing-Tests -test av -trigger-device labjackt4 -labjack-host 192.168.1.100
```

`-trigger-pin` is 1–8 on all five, but it names a different thing on each. Read what the program prints at start-up and probe *that* pin:

- **dlpio8** — the number printed on the terminal block.
- **parallel** — a data line: pin 1 is D0, which is **DB25 pin 2** (D0–D7 are DB25 pins 2–9). Ground is any of DB25 pins 18–25.
- **gpio** — a *position in `-gpio-pins`*, not a line number. With the default list `17,27,22,5,6,13,19,26`, pin 1 is **BCM 17**.
- **ft232h** — a D-bus line counted from 0: pin 1 is **AD0**, pin 8 is AD7. The C-bus (AC0–AC7) is the input side and is not driven here.
- **labjackt4** — a position in DIO4–DIO11: pin 1 is **DIO4**, the screw terminal marked **FIO4**; pin 8 is DIO11 = EIO3 on the DB15. DIO0–DIO3 are the analog inputs AIN0–AIN3 and cannot be driven, which is why the group starts at DIO4.

Prerequisites on Linux: `parallel` needs `sudo modprobe ppdev` and membership of the `lp` group; `gpio` needs kernel $\geq$ 5.10 and membership of the `gpio` group; `ft232h` is Linux-only and needs `ftdi_sio` unloaded (`sudo rmmod ftdi_sio`) plus rw access to `/dev/bus/usb/BBB/DDD` (udev rule or the `plugdev` group); `labjackt4` needs the T4 reachable on TCP port 502. `usermod` changes need a re-login. Verify the wiring with `tests/test_parallel_port`, `tests/test_linuxgpio`, `tests/test_ft232h` or `tests/test_labjackt4` before spending a long capture.

Whichever device is used is written into the results header as `trigger=…`, because they do not yield the same onset-vs-TTL figure. A session that does not record it cannot be compared with one that does.

> **A device that fails to open does not stop the run.** It logs a warning, disables triggers, and continues — so the visual measurement is still obtained. The trap is that the run then exits *successfully* with no TTL in the trace. Under `BBTK_CAPTURE=1` that spends a capture window on a recording with an empty TTL channel. Watch the start-up lines.

Recording it automatically

`tests/Timing-Tests/run-timing-tests.sh` can drive `bbtk-capture` so the stimulus always falls inside the capture window, without two terminals and a stopwatch:

```
BBTK_CAPTURE=1 BBTK_CAPTURE_BIN=~/00_git/bbtkv3/_build/bbtk-capture \
    ./run-timing-tests.sh av-visual
```

It launches the capture, blocks until the device is *actually* recording, runs the stimulus, then waits for the capture to download and save. Only the photodiode steps are wrapped. `BBTK_MARGIN_S` (default 8) sets how much is recorded either side of the stimulus.

The waiting matters: `bbtk-capture` needs 11–40 s between launch and the device recording — fixed command pacing plus an internal-memory erase whose duration depends on whether the box needs a full format — and its own "Capturing events…" message appears several seconds *before* that instant. The script synchronises on the `BBTK-CAPTURE-READY` line emitted the moment the device is armed. Budget that startup for every recorded step.

Microphone coupling

If you are measuring audio, **max the output volume and place the BBTK microphone within a few millimetres of the speaker membrane.** Anything less and events are silently dropped: there is no warning, and the run looks like it worked.

Two checks on the resulting `-events.csv`:

- **N(Mic1) should equal N(TTLin1).** Far fewer means bad coupling, not bad timing — a 197-cycle run yielding 6 Mic events is a placement problem.
- **Mic duration should be close to `frames-on × frame period`** (200 ms at the defaults). A 20 ms Mic pulse for a 200 ms tone means the threshold is catching only the loudest fraction of the tone.

`vrr` — arbitrary stimulus durations

```
Timing-Tests -test vrr -vrr-max-ms 50 -cycles 5
```

Why fixed refresh is a problem On a 60 Hz monitor every duration is a multiple of 16.67 ms. You can show a word for 16.67, 33.33, or 50 ms, but not 20 or 25 ms. At 120 Hz the quantum shrinks to 8.33 ms — better, but still coarse for subliminal work where you may want 10, 12, or 15 ms.

Variable Refresh Rate — AMD FreeSync, NVIDIA G-Sync, or the underlying VESA Adaptive-Sync standard — removes the quantisation. The display holds the current frame for exactly as long as you ask, then refreshes when told. Durations become controlled by your software timer rather than the display clock.

How the test works goxpyriment normally presents with VSync enabled, blocking until the next fixed edge. The `vrr` test switches the renderer to vsync=0 for the duration of the test (restored on exit), then sweeps target durations from 1 ms to `-vrr-max-ms` in 1 ms steps, holding each with a busy-wait and recording what was achieved.

Reading the result

- **On a working VRR display:** duration error stays small and roughly constant across the whole sweep.
- **On a fixed-refresh display:** the error is *periodic* — near zero at multiples of the frame period, rising in between, because the panel can only change on its own schedule. A sawtooth in the per-step output means you do not have VRR, whatever the monitor's box claimed.

The test prints one line per target plus an aggregate over the whole sweep.

```
# Does the connector support it? Look for "vrr_capable: 1"
sudo cat /sys/kernel/debug/dri/0/*/vrr_capable
```

Enabling VRR on Linux Confirm with the test itself rather than trusting the setting — that is the point of having it.

`rt` — reaction-time timestamp precision

```
Timing-Tests -test rt -cycles 50
```

Flashes the screen at a jittered interval and waits for a key press, recording the SDL3 hardware event timestamp against the flip timestamp. This characterises the *input* path: how much delay and jitter your keyboard, USB stack, and event queue add.

Press as fast as you can, or better, use a solenoid — you are characterising the machine, and human variability swamps it. The useful number is the SD, not the mean.

Interpreting results: what is “good”?

Metric	Excellent	Acceptable	Problematic
Frame-interval SD (`display`)	< 0.1 ms	< 0.5 ms	> 1 ms
Frames > 0.5 ms late (`display`)	< 1 %	< 5 %	> 10 %
Audio pipeline latency (`latency`)	< 15 ms	< 30 ms	> 50 ms
Bright-phase duration SD (`av`)	< 0.2 ms	< 1 ms	> 2 ms
TTL→photodiode SD (`av`, hardware)	< 0.5 ms	< 2 ms	> 5 ms
Dropped frames per 100 cycles (`av`)	0	< 5	> 10
VRR duration error SD (`vrr`)	< 0.1 ms	< 0.5 ms	> 1 ms (or periodic → no VRR)
RT SD (`rt`)	< 3 ms	< 10 ms	> 20 ms
Pacing verdict (`display`)	vblank, or blocks	MIXED	does NOT block

The hardware rows are the ones worth quoting in a methods section.

The pacing row grades *what the onsets are anchored to*, not frame quality, and it is the one row where “problematic” still leaves durations and reaction times intact — read The `Frame pacing` block before acting on it.

But an SD alone cannot grade a TTL→photodiode series, and the row above is the one most likely to mislead. A delay that slides steadily across a run has an SD like any other, and a large one — yet nothing about it is jitter, it cannot be averaged away, and every within-channel statistic in the BBTK report looks perfect while it happens. Measured on a Raspberry Pi 4: SD 4.07 ms, of which **99.8 % was a monotonic 13.9**

ms ramp and 0.18 ms was real scatter. Run `timing-drift` (below) before reading this row.

`timing-drift` — is the flip timestamp tracking the panel?

The two `av` output files each look fine on their own; the failure only exists *between* them. This tool joins them and reports the slope first:

```
make timing-drift
./_build/timing-drift bbtk-av-001-events.csv Timing-Tests_sub-000_date-…csv
```

It prints, for TTL→light and then for flip-timestamp→light:

- the **slope**, in µs/cycle and ppm — the number that matters;
- the **de-trended SD**, the real trial-to-trial scatter once the ramp is gone;
- how much of the variance the ramp accounts for;
- one-frame jumps, and the panel's **true frame period** fitted from the photon train, scored against both rates the framework recorded.

The host and the instrument run off different crystals — tens of ppm apart before anything interesting has happened — so the tool fits that clock ratio out of the trigger-versus-flip relation rather than assuming it away. Events are paired by nearest neighbour within half a cycle, not by index, because a single spurious detection shifts every later pair by a whole cycle and turns the series into a constant near minus one period.

Every slope is fitted over the longest stretch containing no dropped frame, and the `fitted over` line says which cycles those were. This is not tidying: a frame-sized step at index k of n points biases a least-squares slope by $6 \cdot h \cdot m \cdot k/n^3$ (with $m = n-k$), which at the midpoint is $1.5 \cdot h/n$. Over a 1000-cycle run at 60 Hz that is 25 µs/cycle — **50 ppm from a single dropped frame in eight minutes**, ten times the effect these captures exist to resolve, and it does not average away with more cycles because the lever arm grows with n too. On a short capture the bias is larger still: 20 cycles with one drop puts it above 2000 ppm.

The `one-frame jumps` count is printed separately, and a run with more than a few of them is not a usable capture however clean the surviving stretch looks — find the load first.

Read the verdict line. `DRIFT DOMINATES` means the flip timestamps and the panel are on different clocks and no absolute onset from that run can be trusted; `NO DRIFT, but … scatter` means the timestamps track the panel and the spread is genuine.

What a validated machine looks like

Two machines, measured with a BBTK v3 photodiode over 1010 cycles each on 2026-08-17, after the flip-timestamp anchoring was corrected:

	Raspberry Pi 4 (V3D/kmsdrm)	Radeon Pro W5700 (radeonsi/X11)
flip → photons, slope	**+0.01 ppm**	**+0.13 ppm**
total drift over the run	0.006 ms / 8.3 min	0.066 ms / 8.3 min
de-trended scatter	0.128 ms	0.278 ms
one-frame jumps	0	0
TTL → light	23.43 ms, SD 0.075 ms (GPIO, kmsdrm)	54.88 ms, SD 0.296 ms (DLP-IO8, **X11**)
TTL → light, same box on kmsdrm	—	**22.55 ms, SD 0.057 ms**
audio-visual lag, 1024 frames	49.88 ms, SD 6.17 ms	23.34 ms, SD 6.54 ms

The TTL row is about the display stack, not the trigger device. The obvious reading — the Pi's GPIO writes through a local ioctl while the W5700's DLP-IO8 crosses a USB link — is wrong, and the third row is the control that shows it. Stopping the X server and running the same machine, panel, photodiode position and DLP-IO8 on bare kmsdrm moved the onset from 54.88 to 22.55 ms and the scatter from 0.296 to 0.057 ms. A USB serial trigger under kmsdrm beats a GPIO trigger under kmsdrm on scatter; the link was never the problem.

The audio row is the same story in reverse — the W5700 drives a USB audio interface and the Pi its on-board codec, which is worth 26 ms of constant latency — while the *jitter* is one audio buffer period over root twelve on both, 6.16 ms predicted at 1024 frames. Nothing about the machine changes it.

The display stack is worth two frames of latency and five times the jitter

W5700, everything else identical	onset latency	scatter
X11, exclusive fullscreen	~55 ms	0.296 ms
kmsdrm, bare console	~22 ms	0.057 ms

Measured 2026-08-17, 481 and 286 trials, one variable changed. 32.4 ms is almost exactly two frames, which is what an X11 Present path plus a driver swapchain queues ahead; on kmsdrm the application page-flips straight to the CRTC.

The panel is not involved: its 10-90% transition measured 16.29 ms on the Pi and 17.28 ms here, with the same asymmetry between halves, so the same monitor is behaving the same way on both machines and the latency is upstream of it.

None of this is visible from inside the program. Both configurations report `class=blocking`, sub-ppm drift against the panel, and frame intervals with sub-millisecond scatter. An experiment that quotes an onset time would be 32 ms wrong under X11 and would have no way to know. If absolute latency matters — EEG, MEG, anything cross-modal — run the stimulus on a console without a display server, and measure it rather than assuming either number.

For the drift row, the W5700 is the more informative case. Its GPU and CPU keep independent crystals, so its loop cadence runs **−4.20 ppm** off nominal; before the anchoring was corrected its flip timestamps advanced on the CPU clock at the nominal rate while the panel ran on the GPU clock, and that difference showed up as a measured **−4.73 ppm** of drift. The two numbers are the same quantity. That the cadence is still −4.20 ppm while the drift is now +0.13 is the evidence that onsets follow the panel rather than the schedule.

Read the run's `sys pacing:` line alongside it

`-test av` and `-test display` both record which branch their presents took, in the `-info.txt` beside `sys vblank_backend`:

```
# sys pacing: presents=30000 blocked=30000 early=29610 paced=0 vblank_held=0 (0.0 % paced)
# sys pacing: presents=30000 blocked=0 early=0 paced=30000 (100.0 % paced) wait_mean=16.14
blocking
```

A drift-free result means two different things in those two cases. **`class=blocking`** means `SDL_RenderPresent` returned at the retrace and every frame re-anchored to the hardware — the pacing schedule was barely used, so the run says little about how accurate that schedule is. **`class=not-blocking`** means the schedule *was* what advanced the clock, and a flat result there is evidence about the schedule itself. `class=vblank` is a third case again: the onsets came from kernel vblank stamps and neither the driver nor the schedule was the reference.

The other fields: `early` is the subset of `blocked` that returned just inside the nominal boundary (see The `Frame pacing` block — normal, and `early_mean` is the phase between the nominal and panel frame grids); `hold_frac` is the share of a frame period the loop spent holding against a synthesised boundary, averaged over every present, and is what `class` is derived from.

Without this line the cases are indistinguishable after the fact, and a set of captures can end up unable to answer the question it was run for.

Loading data in Python

Each run writes two files to `~/goxpy_data/`, or to `-outdir` if given (`run-timing-tests.sh` points it at the session directory so everything from one session stays together):

- `Timing-Tests_sub-000_date-<YYYYMMDD>-<HHMMSS>.csv` — data rows, no comments
- `...-info.txt` — session metadata: start/end time, hostname, OS, and the display and audio configuration actually in use

```
import pandas as pd

df = pd.read_csv("~/goxpy_data/Timing-Tests_sub-000_date-20260728-091541.csv")
```

```
# av: drop the warm-up cycles before quoting anything
steady = df[df.cycle >= 10]
print(steady.bright_duration_ms.describe())
print(steady.period_ms.std())
```

The av test writes one row per cycle: `cycle`, `t_visual_before_ms`, `t_visual_after_ms`, `bright_duration_ms`, `period_ms`, `t_audio_queued_ms`, `soa_intended_ms`, `soa_actual_ms`.

Always read display parameters from the run's own `-info.txt`. On a multi-monitor machine the displays differ in resolution, refresh rate and pixel density, and the scan-out calibration is per-display. The header records which one the window actually opened on.

Improving timing on your system

Linux

- **Disable the compositor.** By far the largest single improvement. Start from a virtual terminal (`Ctrl+Alt+F2`) with the window system stopped (`systemctl stop gdm`) and `SDL_VIDEODRIVER=kmsdrm`. On the reference rig this took dropped frames from ~5 % to zero and removed a frame of latency. Failing that, use a plain window manager (i3, openbox).
- Disable CPU frequency scaling: `cpupower frequency-set -g performance`.
- **Run with real-time scheduling.** Grant your user the privilege once (below); after that, goxpyriment programs — Timing-Tests included — request priority 50 at startup on their own, so nothing needs prefixing:

  ```
  Timing-Tests -test display -duration-s 30       # asks for real-time itself
  Timing-Tests -no-realtime -test display         # opt out
  chrt -f 50 some-other-program                   # for anything that does not
  ```

  This matters most when a program is launched by clicking its icon, where there is no command line to prefix. If the grant is missing the program says so and continues at normal priority rather than refusing to run, and the `Sched:` line in its system report records which it got. The privilege comes from a file you add to `/etc/security/limits.d/` — **not** from editing `/etc/security/limits.conf`, which is package-managed and can be replaced on upgrade, taking your setting with it. See Setting priority under Linux for the procedure: create a group, drop in `/etc/security/limits.d/99-goxpyriment.conf`, add yourself, log out and back in.

  Two traps worth knowing before you spend a session on it:

  - **The limit is inherited from your login session.** A new terminal will not pick up the change; only a full logout and login will. Check `ulimit -r` before trusting a run — if it still says `0`, the grant is not in effect and `chrt` will simply fail.

  - **Editing the file without `sudo` fails silently in some editors.** Confirm the text actually landed, with `grep rtprio /etc/security/limits.d/*.conf`, rather than assuming the save worked.

  Do not reuse an existing group such as `audio` for this. Its rtprio grant belongs to another package (jackd), which can revoke it on reconfigure — and a run that quietly loses real-time priority looks exactly like a run that had it.

  The priority must not exceed the `rtprio` value granted in the setup above. Asking for more fails with "Operation not permitted" – the same error as having no grant at all, so it is easily misdiagnosed. The setup document grants 50, hence `-f 50` here; keep the two numbers equal.

  You *can* skip the setup with `sudo chrt -f 50 Timing-Tests …`, but the test then runs as root: data files land owned by root and it picks up root's environment, not yours. Fine for a one-off check, wrong for collecting data.

- Consider a real-time kernel — see https://ubuntu.com/blog/enable-real-time-ubuntu.
- **Lower the FTDI latency timer** if you read responses over a USB serial device — see the DLP-IO8 section below. Note it does *not* reduce the latency of *sending* a trigger; that is USB frame scheduling, which the timer does not touch.
- **Send triggers off the USB bus if you can.** Since the timer above cannot help with output, the fix is a different device: `-trigger-device parallel` on a machine with an LPT port, or `-trigger-device gpio` on a single-board computer. Both write via one `ioctl`. See Choosing a TTL output device.

Windows

- Disable "Hardware-Accelerated GPU Scheduling" in Display Settings if you see high frame jitter.

Audio buffer size

The hardware audio buffer is controlled by the SDL hint `SDL_AUDIO_DEVICE_SAMPLE_FRAMES`, exposed as `-audio-frames`. It must be set before the audio device opens.

```
# Default (platform-dependent, often 512–2048 frames)
Timing-Tests -test latency

# Aggressive low-latency (~5.8 ms at 44100 Hz)
Timing-Tests -test latency -audio-frames 256 -drain-reps 20

# Conservative (~46 ms, stable anywhere)
Timing-Tests -test latency -audio-frames 2048
```

On startup the program prints the actual device format:

```
audio: 44100 Hz  2 ch  256 sample frames (~5.8 ms latency)
```

The buffer sets the floor on audio-onset precision: onsets can only land on buffer boundaries, so 256 frames at 44100 Hz quantises them to 5.8 ms steps. Use `latency` at several sizes to find the smallest that drains stably (low SD), then use that everywhere — including in your actual experiment.

Choosing the buffer: jitter against glitches

"Stable" above means more than a low SD in `latency`, and the two ends fail in opposite ways. Measured on a Raspberry Pi 4 (PipeWire, 48 kHz) on 2026-08-17, capturing the Pi's line output directly with an Analog Discovery 3 at 100 kS/s and taking the onset from a running-RMS envelope:

driver	`-audio-frames`	period	n	median AV lag	SD	`period/sqrt(12)`	torn (12 s)
PipeWire	2048	42.67 ms	60	101.50 ms	12.28 ms	12.32 ms	none
PipeWire	**1024**	21.33 ms	481	**49.88 ms**	**6.17 ms**	6.16 ms	**none**
PipeWire	512	10.67 ms	500	27.30 ms	7.28 ms*	3.08 ms	10 (2.0 %)
PipeWire	256	5.33 ms	483	9.43 ms	2.36 ms	1.54 ms	100 (20.7 %)
ALSA direct	512	10.67 ms	463	4.95 ms	6.74 ms	3.08 ms	**463 (100 %)**

* inflated by a mid-run escalation, below; its first half scatters by 3.10 ms.

1024 is the recommended setting on this hardware — the only one that is both clean and stable. Below it the stack degrades quickly, and at 512 it does something worse than degrade.

A large buffer costs jitter, and the jitter is a beat, not noise The tone is handed over on time and then waits for the queue, so the audio onset scatters across one buffer period — matching `period/sqrt(12)` to within 0.2 % at 1024 and 2048. But it is not random. The tone's position in the buffer grid advances by `cycle mod period` every trial and wraps, a deterministic sawtooth:

```
cycle 499.830 mod buffer 10.667 = 9.163 ms
  predicted  +1.503 ms per trial, wrapping by -9.163
  observed   +1.500 ms per trial (n=161), wraps -9.170 (n=58)
```

Agreement to 7 us. Two consequences: a per-trial lag can be **predicted** from the cycle, the period and the trial index rather than merely bounded; and a cycle chosen as a whole multiple of the buffer period removes the beat altogether. At 512, a 501.33 ms cycle (47 x 10.667) would hold the lag constant instead of spreading it over 10.7 ms.

A small buffer costs glitches — and PipeWire moves the goalposts At 512, ten of the first 245 tones were torn (gaps to 37.7 ms), and then at trial 245 the lag stepped by **+10.85 ms — one buffer period — and the tearing stopped dead**, with no glitch in the remaining 255 tones. That is the audio server adding a period to the graph after repeated underruns.

So a small buffer is not a setting the machine holds: it underruns, relocates your latency mid-run, and the naive linear fit through that step reports a 15 ms "drift" that does not exist. Anything measured across such a step is two populations averaged together.

The floor is the hardware, not the audio server It is tempting to read the rows above as PipeWire's fault and reach for a "direct" path. That was tested: PipeWire stopped (services *and* sockets, or socket activation restarts it), `SDL_AUDIODRIVER=alsa` and `SDL_AUDIO_ALSA_DEFAULT_PLAYBACK_DEVICE=hw:2,0` to bypass the `default` device, which on a PipeWire system is the PipeWire plugin and fails with `ENOTSUPP` once the server is gone.

Direct ALSA at 512 gave the lowest latency of anything measured — a median lag of **4.95 ms**, with some trials negative, the sound reaching the jack before the panel was 10 % lit. It was also completely unusable: **every one of 463 tones was torn**, typically five times each, the envelope collapsing to about 1 % of plateau every 30-40 ms and recovering.

So the Pi cannot sustain a 10.7 ms buffer, and the audio server was never the constraint. PipeWire's mid-run escalation at 512 was it discovering the same hardware floor and working around it, which is why its glitch rate there was 2 % rather than 100 %. A server that adapts is a nuisance for a measurement and a kindness for an experiment; the fix is to sit above the floor deliberately rather than to remove the thing that was hiding it.

Restart the server afterwards (`systemctl --user start pipewire.socket pipewire-pulse.socket wireplumber.service`), and delete any `~/.asoundrc` written for the test, or the machine will bypass PipeWire from then on.

An instrument's detector can set the scale of what it reports The 512 case was first measured through a BBTK microphone channel, which reported 23 % of tones torn by gaps whose median was 22.3 ms. The electrical capture finds the tearing to be real but 2.0 % of tones with gaps from 0.5 to 37.7 ms, mostly around 2 ms. A threshold detector watching an acoustic envelope needs the signal to recover past its threshold before it calls the tone present again, so short electrical glitches read as long acoustic gaps.

The rate differed too, and plausibly for a real reason: the BBTK session ran `bbtk-capture` on the same Pi, competing for it. Take the lesson as: **a second instrument measuring by a different route is the only way to learn what the first one is adding**, and prefer the route with fewer transducers in it when the question is about software.

What survived every recheck: **scheduling priority is not involved.** The 2x2 of

`512/2048` against `-realtime-priority 50/0` scratched at 512 under both policies and was clean at 2048 under both.

Neither failure appears in anything the host prints — in these same runs the software-side SOA read **0.080 ms +- 0.035**, because handing the tone over on time is the part the software does correctly.

Check underruns at the source rather than by ear: run `pw-top` over ssh (a fullscreen test covers the console) and watch the **ERR** column.

Walkthrough: a Raspberry Pi, GPIO triggers, and BBTK capture

A full `run-timing-tests.sh` session on a Raspberry Pi, driving the TTL from the GPIO header instead of a USB trigger box and recording it on a Black Box ToolKit. The GPIO path matters here: it removes the USB link from between the flip and the TTL edge, so what remains in the onset-vs-TTL spread is the Pi's display path rather than a serial transaction.

> **None of the steps below have been run on a Pi by the author of this section.** The procedure follows from the flags and the wiring; the numbers it produces are what the session is *for*. Treat any figure quoted from another machine in this document as inapplicable until you have measured this one.

1. Find the GPIO chip that owns the 40-pin header

Do not assume `/dev/gpiochip0`. Which chip carries the header varies by Pi model and kernel — on some combinations the header is *not* chip 0, and an unrelated chip is. Ask the system:

```
sudo apt install gpiod        # once
gpiodetect                    # lists chips, line counts, and labels
gpioinfo | less               # per-line names; the header lines are named GPIOnn
```

Pick the chip whose lines are the header's, and pass it as `GPIO_CHIP`. Getting this wrong is not merely inert — another chip's lines may drive real board hardware.

2. Grant access and verify the wiring

```
sudo usermod -aG gpio $USER   # then log out and back in
go run ./tests/test_linuxgpio -chip /dev/gpiochip0 -out 17,27,22,5,6,13,19,26
```

Confirm on a scope or logic analyser that the pin you intend to use actually toggles **before** going any further. `test_linuxgpio` exists precisely so that a wiring fault is found here rather than in a 1000-cycle capture.

3. Wire the TTL and the photodiode

- **TTL** — the header pin for the line you chose, into a BBTK TTL input. With the default `GPIO_PINS`, `TRIGGER_PIN=1` is **BCM 17** (physical pin 11). Ground to any

header ground pin. `Timing-Tests` prints the BCM number it resolved to at start-up — probe that one, not the flag value.

- **Photodiode** — on the **top-left** stimulus square, at the square's centre. The panel scans top-to-bottom, so the bottom squares light nearly a frame later; a second diode on a bottom square measures that gradient.

  3.3 V is not TTL. Pi GPIO swings 0–3.3 V. Whether a given BBTK input latches at 3.3 V is a property of that unit — check it with a 20-cycle run and confirm `N(TTLin1)` equals the cycle count before committing to 1000 cycles. If it does not latch, a 3.3 V→5 V level shifter on the trigger line is the fix; do not feed 5 V back into a Pi input.

4. Get off the desktop

This is worth more than any flag in this document. On the reference hardware the compositor accounted for essentially all of the measured jitter. Run from a bare console with no desktop session:

```
sudo systemctl set-default multi-user.target && sudo reboot   # revert with graphical.t
```

Then, in the console, use the KMS/DRM video driver:

```
export SDL_VIDEODRIVER=kmsdrm
```

Confirm the refresh rate the Pi is actually driving before trusting any duration — `-frames-on 12` is 200 ms only at 60 Hz:

```
./Timing-Tests -test display -duration-s 30
```

5. Run the session

```
cd tests/Timing-Tests
go build .

TRIGGER_DEVICE=gpio \
GPIO_CHIP=/dev/gpiochip0 \
GPIO_PINS=17,27,22,5,6,13,19,26 \
TRIGGER_PIN=1 \
REFRESH_HZ=60 \
BBTK_CAPTURE=1 \
BBTK_CAPTURE_BIN=~/00_git/bbtkv3/_build/bbtk-capture \
    ./run-timing-tests.sh
```

Check the banner before answering the first prompt — it echoes the trigger device, chip and pins, the tone frequency, and the presented/analysed cycle split. If it says `dlpio8`, the environment variable did not take.

Set `BBTK_PORT` if the capture tool cannot find the box on its own. Budget 11–40 s of device setup per recorded step, and remember every `av` step is ~8½ minutes of stimulus at the defaults.

To pilot the wiring cheaply first, shorten the run rather than skipping it:

```
CYCLES=30 WARMUP=5 TRIGGER_DEVICE=gpio ./run-timing-tests.sh av
```

6. Check the capture before believing it

On the resulting `-events.csv`:

- **`N(TTLin1)` should equal the number of cycles presented** (`CYCLES`, including the warm-up ones — they fire the TTL too). Fewer means the 3.3 V swing is not latching, not that triggers were dropped.
- **`N(Opto1)` should equal `N(TTLin1)`.** A shortfall is photodiode placement or threshold, not timing.
- **Opto duration ≈ 200 ms** at the defaults. Note that BBTK smoothing adds roughly 20 ms to raw durations — read `CorrectedDuration`, and do not interpret the raw figure as panel behaviour.
- **The results header should read `trigger=gpio chip=… line=17 ….`** If it says `trigger=none`, the chip failed to open and the run continued without triggers; the capture's TTL channel is empty and the step must be re-run.

7. What the Pi will not fix

A previous Pi session measured a TTL→Opto lag of **38.5 ms**, against the 4–7 ms range Bridges et al. (2020) report across packages. A local GPIO write removes the USB serial link from that figure, but not the Pi's HDMI/display path, which is the more likely explanation for a lag of that size. Expect the GPIO change to tighten the *spread*; do not expect it to close the *lag*. Distinguishing the two requires the raster-position and display-path measurements, not a different trigger.

Related tests

- `tests/test_gv_sync` — `.gv` video playback: does `PlayGvFunc` put a frame on screen when it says it does?
- `tests/test_linuxgpio` — GPIO character-device output/input, used to verify header wiring before a session.
- `tests/test_parallel_port` — LPT data lines via ppdev, the equivalent check for a parallel-port trigger.
- `tests/test_dlpio8` — square-wave output for characterising the trigger box itself against an oscilloscope.
- `tests/test_clear_only_frames` — regression test for the compositor presentation bug described at the top of this document.
- `tests/test_vsync_blocking` — does `SDL_RenderPresent` block on VSYNC on this platform, or return immediately?

DLP-IO8

Driver: `triggers/dlpio8.go`. Full protocol notes and raw data at https://github.com/chrplr/dlp-io8-g; the timing figures below were measured there with a Siglent SDS1104X-E and are repeated because they change how you should use the device.

Lower the FTDI latency timer before reading anything

The DLP-IO8 is an FTDI device, and the `ftdi_sio` driver defaults to a **16 ms latency timer**: the chip holds a partly-filled buffer that long before sending it to the host. A poll the module answers instantly still takes 16 ms to come back.

Measured, n=300 per setting, for an 8-channel read:

`latency_timer`	round trip	poll rate
16 (default)	**15.98 ms**	**63 Hz**
4	3.99 ms	251 Hz
1	1.01 ms	995 Hz

The relationship is exactly `round trip = latency_timer`, so the module's own processing is negligible and the whole cost is driver batching. A polling loop gets the worst case rather than the average: waiting for each reply synchronises the loop to the timer and pays the full 16 ms every iteration.

```
echo 1 | sudo tee /sys/bus/usb-serial/devices/ttyUSB0/latency_timer
```

That reverts on replug. To make it stick:

```
# /etc/udev/rules.d/99-ftdi-latency.rules
SUBSYSTEM=="usb-serial", DRIVERS=="ftdi_sio", ATTR{latency_timer}="1"
```

The rule applies to every FTDI serial device on the machine, not just this one. That is usually what you want, but it is worth knowing it is system-wide.

It does nothing for sending triggers. Output latency is governed by USB frame scheduling. Do not expect this setting to make a trigger arrive sooner.

A multi-bit code is not atomic

There is no multi-channel command: every command is one ASCII byte affecting one line. `Send(mask)` therefore emits eight bytes which the module acts on as they arrive, and **the port takes ~610 µs to settle**, showing partly-updated values throughout. Measured n=99: 86.2 µs per byte, 609.5 µs from the first line to the eighth.

Against a system sampling at 1 kHz that is about 61 % of a sample period, so a code change is sampled mid-transition roughly three times in five and recorded as a value that was never sent.

Use one line per event type, pulsed. A single line is one command byte, so there is no skew at all, and eight lines still distinguish eight event types. Reserve `Send` for a multi-bit code for the cases where the acquisition reads the code milliseconds after

the onset edge rather than latching it at the edge, or where a strobe line is raised last once the code has settled.

Pulse width is only as good as your scheduling

The device has no pulse timer, so a pulse is two host writes and the width inherits host scheduling in full. Measured n=50 per width:

host state	median error	spread
idle	−10 to −20 µs	≤ 120 µs
under CPU load	up to +1.85 ms	up to 4.75 ms

The host's own busy-wait interval degrades by the same amount, tracking the wire to within 80 µs — so the cause is the scheduler descheduling the process, not the USB path or the device. See Setting priority under Linux; that is the fix, and it is a different mechanism from the latency timer above.

Parallel port and GPIO alternatives

Both avoid the USB path entirely: a write is one `ioctl`, not a USB serial transaction subject to frame scheduling. On hardware that offers either, this is the cheapest available improvement to onset-vs-TTL precision.

In the timing tests, select them with `-trigger-device` (see Choosing a TTL output device):

```
Timing-Tests -test av -trigger-device parallel
Timing-Tests -test av -trigger-device gpio -gpio-chip /dev/gpiochip0
```

or, for a whole session, `TRIGGER_DEVICE=parallel` / `TRIGGER_DEVICE=gpio` with `run-timing-tests.sh`.

In your own experiment code:

```
// Parallel port (LPT), 5 V. Needs `sudo modprobe ppdev` and the `lp` group.
p := triggers.NewParallelPort("/dev/parport0")
if err := p.Open(); err != nil { log.Fatal(err) }
defer p.Close()
p.Send(0x01)                    // all 8 data lines at once, D0 = DB25 pin 2

// GPIO character device (Raspberry Pi, Rock Pi, …), 3.3 V. Needs kernel >= 5.10
// and the `gpio` group. Check which chip owns the header with `gpiodetect`.
g, err := triggers.NewLinuxGPIOTrigger(
    triggers.WithGPIOChip("/dev/gpiochip0"),
    triggers.WithGPIOOutputPins([8]int{17, 27, 22, 5, 6, 13, 19, 26}),
)
if err != nil { log.Fatal(err) }
defer g.Close()
g.Pulse(0, 5*time.Millisecond)    // line 0 = the first pin in the array = BCM 17
```

`Send(mask)` sets all 8 lines simultaneously on both — unlike the DLP-IO8, where a multi-bit code is written one byte per line and takes ~610 µs to settle.

Note the voltage difference: parallel is 5 V, GPIO is 3.3 V. Confirm your acquisition system latches at 3.3 V before relying on the GPIO path.